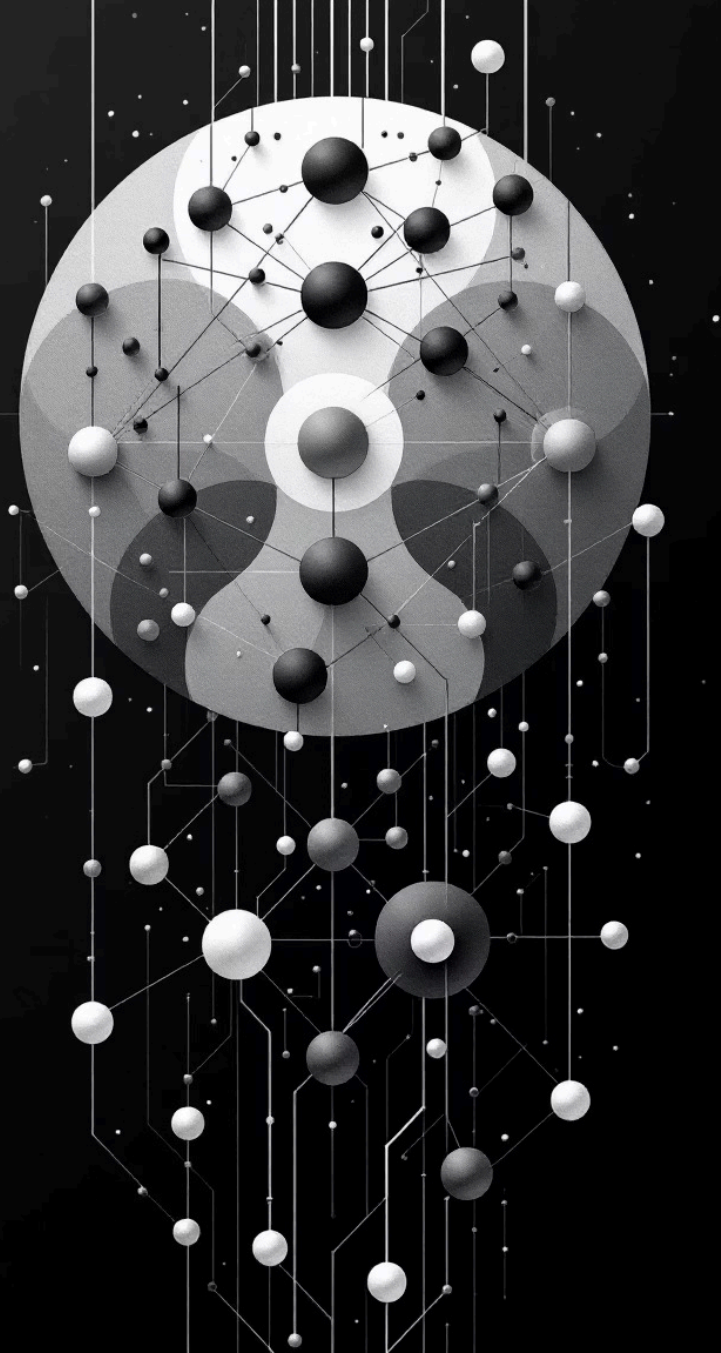


An abstract graphic on the left side of the slide. It features a large, light gray circle containing several smaller, overlapping circles in white and dark gray. These circles are interconnected by a network of thin, light gray lines. Numerous small, solid spheres in white and dark gray are scattered throughout the composition, some appearing to be part of the network structure. The background is a solid dark gray.

Continuous Bag of Words (CBOW) Model

A powerful neural network model from the Word2Vec family that revolutionised natural language processing by learning meaningful word representations through context prediction.

What is CBOW?



Neural Network Foundation

Part of the Word2Vec family introduced by Mikolov et al. in 2013, CBOW uses a shallow neural architecture to learn word relationships.



Context-Based Prediction

Predicts a target word based on its surrounding context words, capturing both semantic and syntactic patterns in language.



Dense Word Embeddings

Learns high-quality word vectors that encode meaningful relationships between words in a continuous vector space.



Wide Applications

Used extensively in NLP tasks including text classification, sentiment analysis, machine translation, and information retrieval.

CBOW vs Traditional Bag of Words

Traditional BoW

- Simply counts word frequency
- Completely ignores word order
- No understanding of context
- Produces sparse, high-dimensional vectors

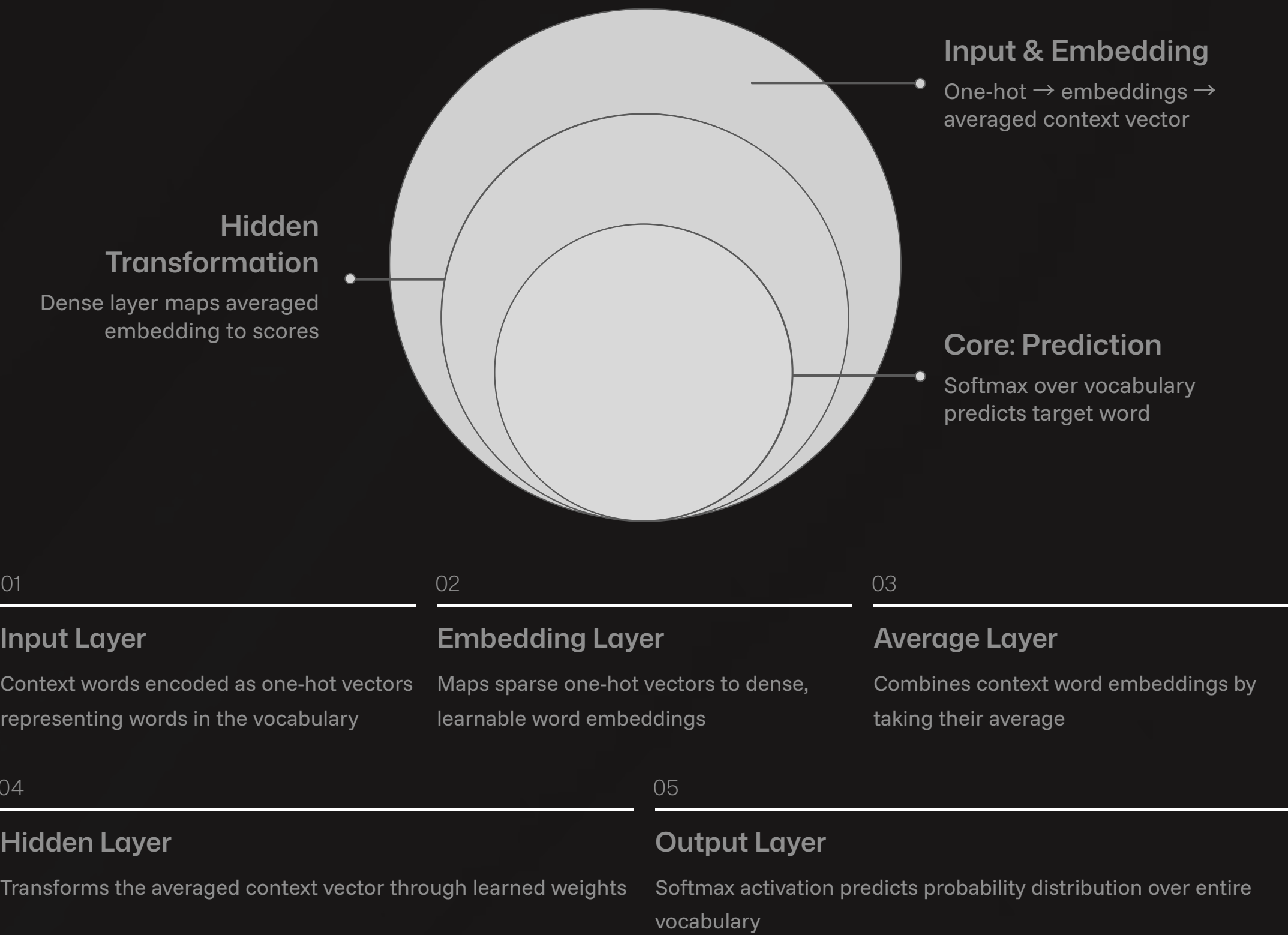
CBOW Model

- Uses context window for prediction
- Captures word order and meaning
- Leverages surrounding context
- Creates dense, meaningful embeddings

CBOW's neural architecture enables it to learn richer word representations that capture semantic relationships missing in traditional approaches.



CBOW Architecture





How CBOW Works: Step-by-Step Example

📄 **Example sentence:** "The cat sat on the mat"

Target word: "sat" | **Context window size:** 2

1

Context Selection

Context words: ["The", "cat", "on", "the"]

Window captures surrounding words

2

Vector Encoding

Convert context words to one-hot vectors

Each word gets unique binary representation

3

Embedding & Average

Map to dense embeddings, then average

Creates single context representation

4

Prediction

Softmax outputs probabilities

Maximise probability of target "sat"

Mathematical Formulation

The CBOW model learns through these key mathematical operations:

Hidden Layer Computation

$$\mathbf{h} = \frac{1}{C} \sum_{i=1}^C \mathbf{W} \mathbf{x}_i$$

Where C is context size, $\mathbf{W}$ is input-to-hidden weight matrix, and $\mathbf{x}_i$ are one-hot context vectors.

Output Scores & Softmax

$$u_j = \mathbf{v}'_{w_j} \cdot \mathbf{h}$$

$$y_j = \frac{\exp(u_j)}{\sum_{j'=1}^V \exp(u_{j'})}$$

The model computes scores for each vocabulary word, then applies softmax for probability distribution.

Training Objective

Minimise negative log-likelihood of correct target word to learn optimal embeddings.

Numerical Example: Predicting "sat"

- 1

Setup Vocabulary

Vocabulary: ["the", "cat", "sat", "on", "mat"] → Size: 5 words

One-hot encoding: "the" = [1,0,0,0,0], "cat" = [0,1,0,0,0]
- 2

Process Context

Context words: "the" and "cat" (simplified window size 2)

Average input vectors: [0.5, 0.5, 0, 0, 0] × Weight matrix
- 3

Calculate Scores

Hidden vector multiplied by output weights produces raw scores

Example scores: [-0.2, 0.1, **1.8**, 0.3, -0.5]
- 4

Generate Probabilities

Softmax converts scores to probabilities

Result: "sat" receives highest probability (≈0.73)

```
# Simplified Python Training Loop
for epoch in range(epochs):
    for context, target in training_data:
        # Forward pass
        h = average_embeddings(context)
        scores = output_weights @ h
        probs = softmax(scores)

        # Compute loss & backpropagate
        loss = -log(probs[target])
        update_weights(loss)
```